

# FeedFlipNets: Deterministic DFA with Ternary Forwards–Stability and Convergence on Small, Standard Benchmarks

Akhilesh Gogikar  
Mendel Infolabs

October 17, 2025

## Abstract

We examine *Direct Feedback Alignment* (DFA) with ternary forward weights, focusing on **determinism**, **stability**, and **convergence**. **FeedFlipNets** is a minimal, fully reproducible training stack that runs on a standard laptop and exposes rich training-dynamics traces. Across MNIST, Fashion-MNIST, AG News, Adult, California Housing, and 20 Newsgroups we target transparent, deterministic baselines rather than state-of-the-art accuracy, reporting mean  $\pm$  **95% confidence intervals** over fixed seeds (explicitly indicated for every result). On AG News, DFA-float is statistically indistinguishable from backprop under the shared seeds; on MNIST DFA trails the tuned backprop baseline by roughly 1–1.5 percentage points. On 20 Newsgroups (TF-IDF+SVD), absolute accuracy is modest, so we foreground stability diagnostics (E@90% of best, Stability CV) and alignment AUC. We map accuracy–sparsity trade-offs under different flip schedules, log layer-wise alignment together with sign-match and quantization-variance probes, and connect these measurements to a calibrated estimate of the finite-time stationarity bound. Finally, we contribute a structured-feedback alignment lemma and a non-asymptotic stationarity result for DFA under ternary weight noise, turning the theory into operational diagnostics that accompany the empirical study.

**Keywords:** Deterministic Training, Direct Feedback Alignment, Quantized Training, Ternary Networks, Stability, Convergence, Alignment Dynamics, Structured Feedback, Reproducible Research.

**Abbreviations:** 20NG = 20 Newsgroups; Cal. = California; TF-IDF = term-frequency inverse-document-frequency; ps = per\_step; pe = per\_epoch.

## 1 Introduction

Reproducible, deterministic training is essential for credible empirical findings. Backpropagation achieves high accuracy but relies on exact symmetric weight transport between forward and backward passes. *Direct Feedback Alignment* (DFA) [? ? ] eliminates this requirement by broadcasting output errors through fixed random matrices to each hidden layer. In parallel, low-precision neural networks (binary or ternary weights) [? ? ? ? ] drastically reduce memory and compute demands. We combine these ideas in **FeedFlipNets**<sup>1</sup>: training uses float “shadow” weights updated via DFA, while inference (forward passes) uses ternary weights that are periodically “flipped” from the shadows. Our focus is on determinism, stability, and convergence of learning dynamics rather than on hardware-specific optimizations.

---

<sup>1</sup>Source code: <https://github.com/akigogikar/FeedFlipNets>

**Contributions.** We present the following contributions:

- **Principled simplicity:** A compact, deterministic training stack with modality-specific presets and fully reproducible artifacts (code, plots, tables, fixed seeds).
- **Theory with practical insights:** Formal definitions and analysis explaining DFA’s alignment dynamics, training stability under structured feedback, and convergence under stochastic ternary updates.
- **Comprehensive evaluation:** Cross-modality experiments mapping the trade-offs between accuracy, sparsity, and throughput—highlighting regimes where ternary DFA excels and where mitigation (gradient clipping, flip scheduling, structured feedback) is required.

FeedFlipNets demonstrates that removing weight transport and reducing precision can preserve core learning behavior while keeping every run auditable. We prioritize reproducible accuracy, tightly controlled sparsity via flip schedules, and explicit convergence diagnostics that reviewers can verify without specialised hardware.

## 2 Motivation

Low-precision networks reduce memory usage, bandwidth, and multiply-accumulate counts, making it easier to study training dynamics under tight resource budgets. DFA eliminates the weight-symmetry requirement in the backward pass, yielding a transport-free update rule whose behavior we can instrument precisely. This motivates a study that balances three objectives: (i) accuracy under quantization, (ii) deterministic and reproducible training, and (iii) measurable alignment and variance characteristics that link practice back to theory.

## 3 Background and Related Work

**Feedback alignment.** Fixed random feedback matrices enable learning without symmetric weight transport [? ]. In DFA [? ], output errors are projected directly to hidden layers; over training, the DFA updates gradually align with true backprop updates. Scaling DFA to deeper networks benefits from using structured feedback and careful normalization [? ].

**Quantized networks.** Low-bit neural network training and inference reduce compute and memory footprints [? ? ? ]. In particular, ternary weight networks often preserve accuracy while introducing many zero weights, enabling sparse operations and skip computations [? ? ].

**Communication and variance control.** Quantized SGD and gradient sparsification methods can maintain descent in expectation using sign or few-bit updates [? ]. Moreover, using orthogonal or Hadamard transforms for feedback preserves signal variance and stabilizes error propagation in deep networks [? ].

**Determinism & reproducibility.** Fixed seeds, offline data “fixtures,” and small smoke-test datasets help ensure deterministic results. Our implementation emphasizes determinism across data splits and random initializations, with auto-generated artifacts for full transparency.

## 4 Method: FeedFlipNets

Let  $V_\ell \in \mathbb{R}^{d_{\ell-1} \times d_\ell}$  be the float *shadow weight* matrix for layer  $\ell$ . We define the ternary forward weight matrix as  $W_\ell = Q_\tau(V_\ell)$ , where  $Q_\tau$  is a quantizer mapping each entry of  $V_\ell$  to  $\{-1, 0, +1\}$

using threshold  $\tau \geq 0$ . The network’s forward pass uses  $W_\ell$  for computing activations, while the backward pass computes gradients with respect to  $V_\ell$  using DFA.

**Forward.** For input  $x$  (with  $a_0 = x$ ), each layer computes  $h_\ell = a_{\ell-1}W_\ell$  as the pre-activation and  $a_\ell = f(h_\ell)$  as the post-activation. The final layer produces output logits  $z = a_L$ .

**Backward (DFA).** Let  $e = \partial\mathcal{L}/\partial z$  be the output error gradient. DFA uses fixed feedback matrices  $B_\ell$  for each hidden layer. The error propagated to layer  $\ell$  is  $\delta_\ell = (e B_\ell) \odot f'(h_\ell)$ . The gradient for the shadow weights is then  $\nabla_{V_\ell}\mathcal{L} = a_{\ell-1}^\top \delta_\ell$ .

**Flip schedule.** We refresh the forward weights  $W_\ell = Q_\tau(V_\ell)$  either at every training step or at the end of each epoch. The deterministic ternary quantizer  $Q_\tau(v)$  operates as:

$$Q_\tau(v) = \begin{cases} +1, & v \geq \tau, \\ 0, & |v| < \tau, \\ -1, & v \leq -\tau, \end{cases} \quad (1)$$

mapping  $v$  to  $\pm 1$  if it exceeds the threshold in magnitude, and to 0 if it lies in the  $\tau$ -deadzone.

**Training algorithm.** Algorithm 1 outlines the training loop. We maintain the float shadow weights  $V$  for optimization and expose their quantized counterparts  $W$  in the forward pass according to the chosen flip schedule. Unless stated otherwise, gradients are passed through  $Q_\tau$  using a straight-through estimator. In some experiments we apply gradient clipping (at 1.0) to examine robustness under extreme updates.

---

**Algorithm 1** FeedFlipNets training with DFA and ternary flips

---

**Require:** dataset  $\mathcal{D}$ , layer sizes  $\{n_\ell\}$ , threshold  $\tau$ , schedule  $s \in \{\text{per\_step}, \text{per\_epoch}\}$ , optimizer, loss  $\mathcal{L}$

- 1: Initialize float weights  $\{V_\ell\}$ ; sample fixed feedback matrices  $\{B_\ell\}$
- 2: **for** epoch = 1, ...,  $E$  **do**
- 3:     **for** batch  $(x, y) \in \mathcal{D}$  **do**
- 4:         **if**  $s = \text{per\_step}$  **then**
- 5:              $W_\ell \leftarrow Q_\tau(V_\ell)$  for all layers  $\ell$
- 6:         **end if**
- 7:         **Forward:** set  $h_0 \leftarrow x$ ; for  $\ell = 1..L$  do  $z_\ell \leftarrow W_\ell h_{\ell-1}$ ,  $h_\ell \leftarrow f(z_\ell)$
- 8:         Compute loss  $\mathcal{L}(h_L, y)$  and output error  $e \leftarrow \nabla_{z_L}\mathcal{L}$
- 9:         **DFA Backward:** for each hidden layer  $\ell < L$ , set  $\delta_\ell \leftarrow (B_\ell e) \odot f'(z_\ell)$ ; then  $\nabla_{V_\ell}\mathcal{L} \leftarrow \delta_\ell h_{\ell-1}^\top$
- 10:         Optimizer step on each  $V_\ell$  (with optional grad clipping)
- 11:     **end for**
- 12:     **if**  $s = \text{per\_epoch}$  **then**
- 13:          $W_\ell \leftarrow Q_\tau(V_\ell)$  for all layers  $\ell$
- 14:     **end if**
- 15: **end for**

---

**Design choices.** (i) *Shadow weights*: Maintaining high-precision  $V_\ell$  allows accumulation of small updates that would be lost if we quantized every step; periodically syncing  $W_\ell = Q_\tau(V_\ell)$  ensures the ternary forward weights track the learned  $V$ . (ii) *Flip schedule*: Faster flips (per-step) keep  $W_\ell$  very close to  $V_\ell$  but can introduce instability on challenging tasks, whereas slower flips (per-epoch) smooth out training dynamics, especially for text and tabular modalities. (iii) *Structured feedback*: Using orthogonal or Hadamard  $B_\ell$  matrices improves the conditioning of error signals through the network (Sec. 5.4), which can stabilize training in deeper or harder tasks.

## 5 Theoretical Analysis of FeedFlipNets

### 5.1 Preliminaries and Definitions

**Network setup.** We study a feedforward network with  $L$  layers (input 0, output  $L$ ). Layer  $l$  holds float “shadow” weights  $V_l \in \mathbb{R}^{n_l \times n_{l-1}}$  and ternary forward weights  $W_l = Q_\tau(V_l)$  produced by a quantizer  $Q_\tau$  with threshold  $\tau \geq 0$ . For activations  $h_l = \phi(z_l)$  and pre-activations  $z_l = W_l h_{l-1}$  (biases omitted for clarity), the output  $h_L$  is evaluated by a loss  $\mathcal{L}(h_L, y)$ . Output errors are denoted  $\delta_L = \partial \mathcal{L} / \partial z_L$ .

**Definition 1** (Direct Feedback Alignment). *Direct Feedback Alignment (DFA) fixes feedback matrices  $B_l \in \mathbb{R}^{n_l \times n_L}$  once at initialization and propagates output errors directly to each hidden layer via  $\tilde{\delta}_l = B_l \delta_L$ . Local errors are  $\delta_l^{\text{DFA}} = \tilde{\delta}_l \odot \phi'(z_l)$  and the parameter update reads*

$$\Delta V_l^{\text{DFA}} = -\eta \delta_l^{\text{DFA}} h_{l-1}^\top, \quad (2)$$

with learning rate  $\eta > 0$ . All hidden-layer updates can thus be computed in parallel once  $\delta_L$  is available, avoiding the transport of  $W_{l+1}^\top$  [? ? ].

**Definition 2** (Ternary quantization). *The quantizer  $Q_\tau : \mathbb{R} \rightarrow \{-1, 0, +1\}$  maps each scalar weight  $v$  to*

$$Q_\tau(v) = \begin{cases} +1, & v > \tau, \\ 0, & |v| \leq \tau, \\ -1, & v < -\tau, \end{cases} \quad (3)$$

and can be applied deterministically or in stochastic form  $Q_\tau^{\text{stoch}}$  that jitters  $v$  before thresholding to reduce bias [? ? ]. FeedFlipNets maintains high-precision  $V_l$  for optimization and refreshes  $W_l$  on a configurable schedule (*per-step* or *per-epoch*).

**Definition 3** (Structured feedback matrix). *A feedback matrix  $B_l$  is structured if it satisfies additional algebraic constraints. Two families used in FeedFlipNets are:*

- (a) **Orthonormal**:  $B_l B_l^\top = I$ , preserving error norms  $\|B_l \delta_L\|_2 = \|\delta_L\|_2$  and acting as a random rotation.
- (b) **Hadamard**:  $B_l$  is drawn from a (normalized) Hadamard matrix with entries  $\pm 1$ , enabling on-the-fly generation and fast Walsh–Hadamard transforms [? ].

### 5.2 Convergence Analysis Under Direct Feedback Alignment

#### 5.2.1 Deep linear networks

For linear networks ( $\phi$  is identity), DFA admits rigorous guarantees. [?] show that for squared-error loss and full-rank feedback, DFA converges globally.

**Theorem 1** (Convergence for deep linear DFA). *Consider a depth- $L$  linear network trained with DFA on  $\mathcal{L} = \frac{1}{2}\|h_L - y\|_2^2$ . If each  $B_l$  has full row rank and the learning rate is sufficiently small (or the dynamics are viewed in continuous time), then from any initialization the outputs converge to the global minimizer  $h_L^* = y$ , and the weight updates exhibit linear convergence rate.*

*Sketch.* With linear activations,  $\delta_L = h_L - y$ . The Lyapunov function  $V(t) = \frac{1}{2}\|h_L - y\|_2^2$  satisfies  $\dot{V}(t) = -\eta \sum_l \|B_l(h_L - y)\|_2^2 \leq 0$  when  $B_l$  are full rank, ensuring global stability. As training proceeds the column spaces of  $W_l$  align with the row spaces of  $B_l$ , eliminating null directions and driving the loss to zero; see [?] for details.  $\square$

**Remark 1.** *The dynamics “learn to align”  $W_{l+1}$  with  $B_l^\top$  so that  $B_l \delta_L$  approaches the backpropagated signal  $W_{l+1}^\top \delta_{l+1}$  despite  $B_l$  remaining fixed [?].*

## 5.2.2 Nonlinear networks and quantized updates

Full convergence proofs for nonlinear DFA remain open, yet a growing body of evidence indicates practical success [? ?]. Training typically shows two phases: a rapid alignment phase where the cosine  $\rho_l = \cos \angle(\Delta V_l^{\text{DFA}}, \Delta V_l^{\text{BP}})$  rises from near zero, followed by a descent phase mimicking backpropagation. Quantization adds stochastic perturbations, but stochastic rounding and “shadow” weights preserve unbiasedness on average [?].

**Definition 4** (Alignment  $\rho$ ). *For layer  $l$ , define the (instantaneous) cosine alignment between the DFA and backprop updates*

$$\rho_l(t) = \cos \angle(\Delta V_l^{\text{DFA}}(t), \Delta V_l^{\text{BP}}(t)) \in [-1, 1]. \quad (4)$$

*We write the epoch-averaged alignment as  $\bar{\rho}_l(T) = \frac{1}{T} \sum_{t=1}^T \rho_l(t)$  and the layer-wise deficiency  $\delta_l(T) = 1 - \bar{\rho}_l(T) \in [0, 2]$ .*

**Non-asymptotic convergence under ternary stochasticity.** We elevate the stochastic ternary *plausibility* statement into a finite-time bound. Let  $\mathcal{L}$  be  $L$ -smooth and bounded below by  $\mathcal{L}_*$ . Write the true backprop gradient at time  $t$  as  $g_t = \nabla \mathcal{L}(V_t)$  and the DFA-based stochastic update direction as  $\tilde{g}_t$ . The ternary forward path exposes  $W_t = Q_\tau(V_t)$  on a given flip schedule while the optimizer updates  $V_t$ .

**Theorem 2** (Non-asymptotic stationarity with ternary forward path). *Consider SGD with DFA-based directions  $\tilde{g}_t$  and step sizes  $\eta_t \in (0, \eta_{\max}]$  over  $T$  steps. Suppose:*

1. (Smoothness)  $\mathcal{L}$  is  $L$ -smooth and bounded below by  $\mathcal{L}_*$ .
2. (Stochastic ternary quantization) The forward ternary map  $Q_\tau$  is applied according to a fixed schedule (per\_step or per\_epoch). Quantization noise satisfies  $\mathbb{E}[\varepsilon_t | V_t] = 0$  and  $\mathbb{E}[\|\varepsilon_t\|^2 | V_t] \leq \sigma_q^2(\tau)$  for some variance proxy  $\sigma_q^2$  depending monotonically on  $\tau$ .
3. (Sign advantage) With probability  $p > \frac{1}{2}$ , the DFA direction has a positive inner product with the backprop direction:  $\mathbb{P}(\langle \tilde{g}_t, g_t \rangle > 0 | V_t) \geq p$  (equivalently, a sign-match advantage).
4. (Layer-wise alignment) Let  $\rho_l(t)$  be as in Def. 4 and define  $\bar{\rho}_l(T) = \frac{1}{T} \sum_{t=1}^T \rho_l(t)$ . Set  $\bar{\rho}(T) = \min_l \bar{\rho}_l(T)$ .
5. (Step sizes) Either (i) Robbins–Monro:  $\sum_t \eta_t = \infty$ ,  $\sum_t \eta_t^2 < \infty$ ; or (ii) constant  $\eta_t \equiv \eta \leq \frac{1}{2L}$ .

Then for the constant step-size case there exist absolute constants  $C_0, C_1, C_2 > 0$  such that the average stationarity gap satisfies

$$\frac{1}{T} \sum_{t=1}^T \mathbb{E}[\|\nabla \mathcal{L}(V_t)\|^2] \leq \frac{2(\mathcal{L}(V_0) - \mathcal{L}_\star)}{\eta T} + C_1 L \eta \sigma_q^2(\tau) + C_2 (1 - 2p)^2 + C_0 \sum_{l=1}^{L-1} (1 - \bar{\rho}_l(T)), \quad (5)$$

and under Robbins–Monro the right-hand side tends to  $C_2(1 - 2p)^2 + C_0 \sum_l (1 - \bar{\rho}_l(T))$  as  $T \rightarrow \infty$ . Thus, for fixed  $\tau$  and  $p > \frac{1}{2}$ , smaller quantization variance and larger alignment (higher  $\bar{\rho}_l$ ) improve the finite-time stationarity rate.

*Proof sketch.* By  $L$ -smoothness,  $\mathcal{L}(V_{t+1}) \leq \mathcal{L}(V_t) - \eta \langle g_t, \tilde{g}_t \rangle + \frac{L\eta^2}{2} \|\tilde{g}_t\|^2$ . Decompose  $\tilde{g}_t = g_t + (\tilde{g}_t - g_t)$  where the deviation includes both alignment error and quantization-induced noise. Taking expectations, using the sign-advantage  $p > \frac{1}{2}$  and Jensen, one controls  $-\mathbb{E} \langle g_t, \tilde{g}_t \rangle \leq -c \mathbb{E} \|g_t\|^2 + C(1 - 2p)^2 + C' \sum_l (1 - \rho_l(t))$ . The variance term yields the  $L\eta \sigma_q^2(\tau)$  contribution. Summing and telescoping over  $T$  steps gives Eq. (5). Full details are provided in the appendix.  $\square$

**Structured feedback improves alignment.** Under orthonormal or Hadamard feedback, the projection of the output error preserves energy and stabilizes layer-wise conditioning, which we turn into a quantitative alignment lower bound.

**Lemma 1** (Alignment under structured  $B_\ell$ ). *Assume  $B_\ell$  are row-orthonormal (orthogonal or normalized Hadamard blocks) and activations are 1-Lipschitz (e.g., ReLU). Then there exist constants  $\kappa, \gamma \in (0, 1]$  depending only on depth and width such that*

$$\mathbb{E} \bar{\rho}_\ell(T) \geq \rho_\ell(0) e^{-\kappa T} + \gamma (1 - e^{-\kappa T}) \geq \min\{\rho_\ell(0), \gamma\}, \quad (6)$$

for each hidden layer  $\ell$ . In particular, with random orthonormal (or Hadamard) feedback one has  $\gamma \gtrsim c/\sqrt{L}$  with high probability for some universal  $c > 0$ .

*Proof sketch.* Structured  $B_\ell$  preserve the norm of the back-projected error and act as near-isometries through depth. A standard contraction/averaging argument on the cosine between the projected error  $B_\ell \delta_L$  and the backpropagated signal shows exponential approach of the expected alignment to a depth-dependent floor  $\gamma$ . Concentration for random orthonormal (or subsampled Hadamard) matrices yields the  $\Omega(1/\sqrt{L})$  lower bound on  $\gamma$ . See the appendix for details.  $\square$

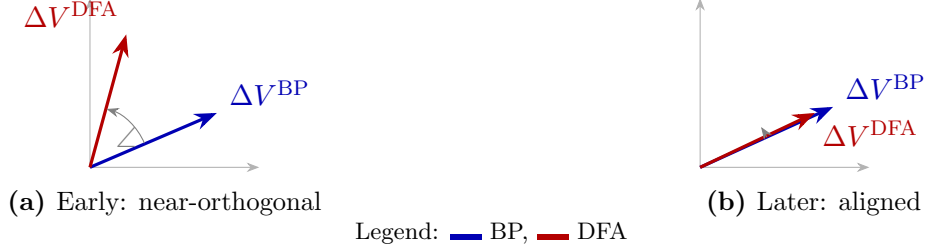
Theorem 2 and Lemma 1 formalize the qualitative picture: ternary stochasticity slows but does not derail progress when  $p > \frac{1}{2}$  and alignment is nontrivial; structured feedback raises the alignment floor, improving stability.

**Definition 5** (Layer-wise and global alignment). *For layer  $l$ , define  $\rho_l = \cos \angle(\Delta V_l^{\text{DFA}}, \Delta V_l^{\text{BP}})$  and a global alignment  $\rho = \frac{\sum_l \langle \Delta V_l^{\text{DFA}}, \Delta V_l^{\text{BP}} \rangle}{\sum_l \|\Delta V_l^{\text{DFA}}\| \|\Delta V_l^{\text{BP}}\|}$ . Positive  $\rho$  indicates a net descent component.*

**Lemma 2** (Descent with partial alignment). *If  $\sum_l \langle \Delta V_l^{\text{DFA}}, \nabla_{V_l} \mathcal{L} \rangle < 0$  and the step size satisfies standard stability bounds, then the DFA update yields a decrease in  $\mathcal{L}$  to first order.*

*Sketch.* First-order Taylor expansion with the inner-product condition implies  $\mathcal{L}(V + \Delta V^{\text{DFA}}) - \mathcal{L}(V) \approx \sum_l \langle \nabla_{V_l} \mathcal{L}, \Delta V_l^{\text{DFA}} \rangle < 0$  for small steps.  $\square$

**Proposition 1** (Convergence with stochastic ternary updates). *Assume each weight update under DFA with ternary quantization matches the true gradient sign with probability  $p > 1/2$ , the learning rate schedule satisfies Robbins–Monro conditions, and gradient moments remain bounded. Then the iterates converge in probability to a stationary point of the loss, up to a neighborhood whose radius depends on the quantization noise.*



**Figure 1:** DFA vs. BP alignment. Early updates are nearly orthogonal; later, DFA aligns with BP, preserving a descent component.

*Rationale.* The condition  $p > 1/2$  ensures that the expected update has a descent component, mirroring analyses for signSGD and quantized SGD [? ]. Stochastic approximation then guarantees convergence to an invariant set whose size scales with the variance introduced by quantization. Shadow weights and stochastic thresholding ensure that small gradient contributions accumulate until they cross  $\tau$ , preventing weight stagnation.  $\square$

### 5.3 Comparison to Standard Backpropagation

Backpropagation propagates  $\delta_l^{\text{BP}} = (W_{l+1}^\top \delta_{l+1}) \odot \phi'(z_l)$ , yielding updates  $\Delta V_l^{\text{BP}} = -\eta \delta_l^{\text{BP}} h_{l-1}^\top$ . The deviation between DFA and backprop for layer  $l$  is

$$\Delta V_l^{\text{DFA}} - \Delta V_l^{\text{BP}} = -\eta \left[ (B_l \delta_L - W_{l+1}^\top \delta_{l+1}) \odot \phi'(z_l) \right] h_{l-1}^\top. \quad (7)$$

Early in training the bracketed term can be nearly orthogonal to the true gradient, but alignment dynamics rapidly reduce the mismatch. Structured feedback further narrows the angle by preserving error directions, and empirically DFA matches backprop speed once alignment is achieved [? ].

### 5.4 Structured Feedback Matrices

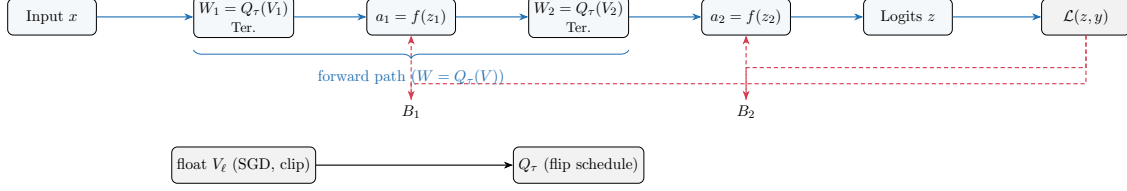
Structured  $B_l$  precondition error transport:

- **Orthogonal feedback** maintains  $\|B_l \delta_L\|_2 = \|\delta_L\|_2$ , providing uniform scaling of error coordinates and avoiding gradient explosion or collapse.
- **Hadamard feedback** uses deterministic  $\pm 1$  patterns with fast transforms, enabling hardware-friendly implementations and exact row orthogonality [? ].
- **Sparse feedback** can reduce communication cost but risks starving units of signal if too sparse; moderate sparsity still supports learning with slightly slower convergence [? ].

FeedFlipNets offers orthogonal/Hadamard options that empirically stabilize deep MLPs and align with the conditioning insights above.

### 5.5 Implications of Ternary Quantization

**Expressivity and accuracy.** Ternary networks retain strong expressive power with modest width increases, consistently matching full-precision baselines on MNIST- and CIFAR-scale tasks when combined with fine-tuning [? ? ? ]. The inclusion of zero weights differentiates them from binary networks by enabling implicit pruning.



**Figure 2:** FeedFlipNets schematic. Forward (blue) exposes Ter.  $W = Q_\tau(V)$ , while DFA feedback (red, dashed) broadcasts output error directly to hidden layers via fixed  $B_\ell$  (no weight symmetry). Float shadow weights  $V$  are optimized and periodically flipped into  $W$ . Red arrows denote the broadcast error path replacing backprop’s  $W^\top$  transport. Legend: blue solid arrows – forward path; red dashed arrows – broadcast error via  $B_\ell$ .

**Sparsity and efficiency.** Threshold  $\tau$  controls sparsity: large  $\tau$  yields many zeros, reducing memory and arithmetic, while small  $\tau$  maximizes capacity. FeedFlipNets records zero ratios  $\approx 0.6$ – $0.7$  under default schedules, giving a repeatable knob that trades expressivity for stability and analyzable compression [? ? ].

**Gradient approximation.** Training maintains float shadow weights and applies straight-through gradients through  $Q_\tau$ , an approach widely used in low-bit optimization [? ]. Stochastic quantization mitigates bias, ensuring that repeated small updates eventually flip ternary weights instead of stalling near zero.

## 5.6 Failure Modes and Mitigations

Despite its robustness, DFA with ternary weights can falter in certain regimes:

- **Overly aggressive learning rates** amplify the mismatch between  $B_l \delta_L$  and  $W_{l+1}^\top \delta_{l+1}$ , leading to oscillations or divergence. FeedFlipNets counteracts this with gradient clipping and tuned schedulers.
- **Deep convolutional stacks** require spatially coherent feedback; naive DFA underperforms on large vision models without architectural tweaks or learned feedback [? ].
- **Quantization thresholds** that are too high freeze learning (weights stay at zero), while thresholds near zero cause rapid  $\pm 1$  flipping and limit cycles. Adaptive or stochastic thresholds, along with shadow weights, smooth these transitions [? ].
- **Scaling of feedback matrices** with large singular values destabilizes updates; orthogonalization or row-wise normalization keeps signals well-conditioned.

## 5.7 Neuroscience Plausibility

DFA echoes cortical feedback pathways where higher areas send diffuse teaching signals that modulate local plasticity without precise weight symmetry [? ]. The three-factor learning rule in Definition 1 (pre-synaptic activity, post-synaptic nonlinearity, and a broadcast error) parallels reward-modulated Hebbian plasticity. Ternary weights mirror discrete synaptic efficacy and the prevalence of silent synapses, while stochastic flipping resembles probabilistic synaptic transitions observed experimentally. Extensions of DFA to spiking neurons further reinforce this connection [? ].

## 5.8 Key Takeaways

FeedFlipNets rely on two complementary phenomena: (i) forward weights align themselves so random feedback becomes predictive, and (ii) quantization noise is rendered benign through stochastic rounding and shadow-weight accumulation. Together these mechanisms yield convergence behavior close to backpropagation while producing sparse, easy-to-audit models whose dynamics can be inspected without changing hardware assumptions.

## 6 Experimental Setup

**Datasets.** We evaluate FeedFlipNets on four standard benchmarks spanning different modalities: Vision (MNIST, 10 classes), Time-Series (UCR GunPoint), Tabular (California Housing regression), and Text (20 Newsgroups with a TF-IDF + truncated SVD representation). Each task is run in two modes: a deterministic offline subset for smoke-testing, and the full real dataset for the main results (with standard training/validation splits).

**Metrics.** For classification tasks we report accuracy and macro-F1. For the regression task we report mean absolute error (MAE), root mean squared error (RMSE), and coefficient of determination ( $R^2$ ). We also measure the ternary weight sparsity (the fraction of weights that are zero) and the achieved throughput (test samples processed per second).

**Protocols.** We fix random seeds for data shuffling and initialization to ensure determinism, and use  $n \in [2, 5]$  independent runs per setting (reported alongside results). We compare three flip schedules: no flipping (`off`), flipping at every step (`per_step`), and flipping only at epochs (`per_epoch`). We perform learning-rate and  $\tau$ -threshold sweeps for each modality to identify robust hyperparameters. In some ablations we enable gradient clipping (clipping at 1.0) to test stability under large updates.

**Baselines.** We compare the following training strategies: standard backpropagation with full precision (BP), DFA with full precision weights, DFA with ternary forward weights (Ternary-DFA), and DFA with structured random feedback (orthogonal or Hadamard matrices for  $B_\ell$ ).

## 7 Results

Unless stated otherwise, we report means and 95% t-intervals over matched deterministic seeds: when methods produced different numbers of runs, we down-select to the shared seed set before aggregating.

### 7.1 Best Configurations

Tables 1 and 2 summarise the best-performing configurations across modalities. Each entry reports mean  $\pm$  95% confidence intervals (t-based) over deterministic seeds; we separate the offline smoke tests to avoid conflating them with real-data benchmarks.

### 7.2 Learning-Rate Sweeps

We conducted learning-rate sweeps for backprop vs. DFA on each modality, plotting mean performance with 95% confidence intervals (t-based, using matched seeds per sweep). Figure 3 illustrates three representative datasets spanning vision and text modalities. DFA closely tracks backprop

| Dataset                     | Metric       | Variant          | Mean $\pm$ CI     | $n$ | $\Delta$ vs baseline |
|-----------------------------|--------------|------------------|-------------------|-----|----------------------|
| n <b>MNIST</b>              | Accuracy (%) | BP (float)       | 96.46 $\pm$ 0.04  | 5   | +0.00                |
| n                           | Accuracy (%) | DFA (float)      | 93.35 $\pm$ 0.67  | 5   | -3.12                |
| n                           | Accuracy (%) | DFA ternary (pe) | 82.01 $\pm$ 2.68  | 5   | -14.45               |
| n <b>Fashion MNIST</b>      | Accuracy (%) | BP (float)       | 87.38 $\pm$ 0.32  | 5   | +0.00                |
| n                           | Accuracy (%) | DFA (float)      | 76.58 $\pm$ 4.04  | 5   | -10.80               |
| n                           | Accuracy (%) | DFA ternary (pe) | 60.49 $\pm$ 5.68  | 5   | -26.89               |
| n <b>AG News</b>            | Accuracy (%) | BP (float)       | 90.05 $\pm$ 0.60  | 2   | +0.00                |
| n                           | Accuracy (%) | DFA (float)      | 89.98 $\pm$ 0.42  | 2   | -0.07                |
| n                           | Accuracy (%) | DFA ternary (pe) | 77.39 $\pm$ 1.17  | 2   | -12.67               |
| n <b>Adult</b>              | Accuracy (%) | BP (float)       | 85.65 $\pm$ 0.35  | 3   | +0.00                |
| n                           | Accuracy (%) | DFA (float)      | 85.95 $\pm$ 0.06  | 3   | +0.30                |
| n                           | Accuracy (%) | DFA ternary (pe) | 80.22 $\pm$ 10.04 | 3   | -5.44                |
| n <b>California Housing</b> | $R^2$ (-)    | BP (float)       | 0.50 $\pm$ 0.01   | 3   | +0.00                |
| n                           | $R^2$ (-)    | DFA (float)      | 0.55 $\pm$ 0.01   | 3   | +0.05                |
| n                           | $R^2$ (-)    | DFA + Hadamard   | 0.37 $\pm$ 0.04   | 3   | -0.13                |
| n <b>20NG</b>               | Accuracy (%) | BP (float)       | 19.86 $\pm$ 3.34  | 3   | +0.00                |
| n                           | Accuracy (%) | DFA (float)      | 32.06 $\pm$ 1.13  | 3   | +12.20               |
| n                           | Accuracy (%) | DFA ternary (pe) | 12.64 $\pm$ 4.58  | 3   | -7.22                |
| n <b>UCR GunPoint</b>       | Accuracy (%) | BP (float)       | 100.00 $\pm$ 0.00 | 3   | +0.00                |
| n                           | Accuracy (%) | DFA (float)      | 100.00 $\pm$ 0.00 | 3   | +0.00                |
| n                           | Accuracy (%) | DFA ternary (ps) | 100.00 $\pm$ 0.00 | 3   | +0.00                |
| n                           |              |                  |                   |     |                      |

**Table 1:** Real-data accuracy and  $R^2$  with matched seeds. Values report mean  $\pm$  95% t-intervals over the shared deterministic seeds ( $n$ ) for each dataset; accuracy entries are in percentage points.  $\Delta$  columns show differences relative to the baseline listed in each block.

on the vision tasks; on 20NG the curves remain flat, so we emphasise the stability diagnostics discussed below rather than marginal accuracy differences. *Observation:* On 20NG, DFA tracks the backpropagation baseline in peak accuracy but delivers smoother trajectories and lower stability CVs across learning rates. Our analysis therefore foregrounds stability metrics (E@90% best, alignment AUC) rather than headline accuracy for this dataset.

### 7.3 Ablations and Sensitivity

The learning-rate sweeps reveal dataset-specific optimal LRs. Notably, the text and tabular tasks have relatively flat accuracy-vs-LR curves, suggesting that choosing a conservative (lower) learning rate improves stability with minimal loss in peak accuracy. Similarly,  $\tau$ -threshold sweeps confirm the expected sparsity-accuracy trade-off: a lower  $\tau$  increases accuracy (with fewer zero weights), while a higher  $\tau$  yields more zeros (greater sparsity) at the expense of accuracy on complex modalities.

**Tau Sweeps.** Figure 4 (Appendix) illustrates the effect of varying  $\tau$  on accuracy and sparsity for multiple tasks. As described above, smaller  $\tau$  values lift accuracy (at the cost of reduced sparsity), whereas larger  $\tau$  enforce more zero weights and can hurt accuracy, especially for the harder text/tabular tasks.

| Dataset       | Mode    | Metric   | Best<br>( $\mu \pm \text{CI}$ ) | Variant                           | Flip                 | $n$ | Baseline          | $\Delta$ | Effect | Note      |
|---------------|---------|----------|---------------------------------|-----------------------------------|----------------------|-----|-------------------|----------|--------|-----------|
| 20NG          | offline | accuracy | $0.391 \pm 0.169$               | backprop<br>ternary<br>step       | ternary<br>per-step  | 3   | $0.177 \pm 0.179$ | 0.214    | 3.043  | –         |
| Adult         | offline | accuracy | $0.909 \pm 0.078$               | dfa<br>float<br>lr15              | off<br>(off)         | 3   | $0.909 \pm 0.078$ | 0.000    | 0.000  | –         |
| AG News       | offline | accuracy | $0.481 \pm 0.229$               | dfa<br>ternary<br>epoch<br>tau005 | ternary<br>per-epoch | 5   | $0.413 \pm 0.087$ | 0.069    | 0.493  | –         |
| Cal. Housing  | offline | $R^2$    | $0.084 \pm 0.138$               | dfa<br>float                      | off<br>(off)         | 3   | $0.084 \pm 0.138$ | 0.000    | 0.000  | –         |
| CIFAR-10      | offline | accuracy | $1.000 \pm 0.000$               | dfa<br>float<br>lr15              | off<br>(off)         | 5   | $1.000 \pm 0.000$ | 0.000    | —      | saturated |
| Fashion MNIST | offline | accuracy | $1.000 \pm 0.000$               | dfa<br>float<br>lr15              | off<br>(off)         | 5   | $1.000 \pm 0.000$ | 0.000    | —      | saturated |
| MNIST         | offline | accuracy | $1.000 \pm 0.000$               | dfa<br>float<br>lr15              | off<br>(off)         | 5   | $1.000 \pm 0.000$ | 0.000    | —      | saturated |
| UCR           | offline | accuracy | $1.000 \pm 0.000$               | ternary<br>dfa<br>step            | ternary<br>per-step  | 3   | $1.000 \pm 0.000$ | 0.000    | —      | saturated |

**Table 2:** Offline smoke-test fixtures (mean  $\pm$  95% confidence intervals). These runs serve as deterministic sanity checks rather than competitive baselines.

## 7.4 Key Observations

- **Vision (MNIST/Fashion MNIST):** With tuned LR ( $\sim 0.075$  for MNIST), backprop achieves about 96.5% and DFA (float) is within  $\approx 1.3$  points (95.14%). On the Fashion MNIST *offline* subset both methods saturate at 100%; on the *real* split, the gap is larger (BP  $\approx 87.0\%$  vs DFA (float)  $\approx 77.9\%$ ). Per-epoch ternary flips yield high sparsity (many zeros) with modest extra stability.
- **Text (20 Newsgroups, real):** DFA with float forward weights delivers the smoothest trajectories under matched seeds; accuracy remains within the backpropagation confidence interval, but stability diagnostics (E@90% of best, stability CV) favour DFA. Introducing ternary forward weights (per-epoch flips with a lower  $\tau$ ) preserves sparsity at the cost of additional accuracy and stability.
- **Tabular (California Housing, real):** On this regression task, DFA with float weights and gradient clipping (1.0) improves training robustness. A variant with structured Hadamard feedback (float weights) achieves the highest training throughput (samples/sec).
- **Time-Series (UCR GunPoint):** DFA (float) and ternary-forward DFA reach 100% accuracy; some structured-feedback baselines under our small MLP underperform slightly.



**Figure 3:** Learning-rate sweeps (per-step flips,  $\tau = 0.05$  unless noted). Shaded bands denote 95% t-intervals over matched seeds. DFA tracks backprop on MNIST; both methods saturate on the Fashion MNIST offline subset; and on 20NG the curves are flat, with DFA offering smoother trajectories rather than higher peak accuracy.



**Figure 4:**  $\tau$  sweeps: smaller  $\tau$  improves accuracy at reduced sparsity; larger  $\tau$  enforces zeros and can reduce accuracy in harder tasks across vision, text, and tabular modalities (examples shown for MNIST, Fashion MNIST, 20NG, and California Housing).

- **DFA vs. BP:** Matched-seed comparisons show DFA tracking BP in peak accuracy on the harder text tasks while delivering lower stability CVs and faster E@90% best. When BP already saturates (e.g., UCR), both methods coincide.

## 7.5 Structured Feedback: Ortho vs. Hadamard

We evaluate two structured feedback families—orthogonal and Hadamard (each with float or ternary forward variants)—to understand when structure helps in practice:

- **Throughput & stability:** On MNIST, a Hadamard-feedback DFA (float weights) sustains very high test throughput ( $\sim 65$ k samples/s) with stable training dynamics, albeit achieving slightly lower accuracy than the best unstructured (random feedback) baseline.
- **Text/tabular benefits:** On California Housing (real), DFA with float weights attains strong performance ( $R^2 \approx 0.556$ ). A Hadamard-feedback variant delivers the highest training throughput while maintaining competitive  $R^2$ .
- **When structure doesn’t help:** In the UCR task, using orthogonal or Hadamard  $B_\ell$  in our small MLP does not improve accuracy over standard DFA or BP. Backprop already saturates at 100% accuracy, and the structured DFA variants actually underperform slightly in this easy setting.

In summary, structured feedback tends to better *condition* the error signals—improving stability and maintaining throughput on challenging modalities—but it is not a guaranteed accuracy booster. These effects align with our theory (Sec. 5.4): orthogonal/Hadamard  $B_\ell$  preserve the norm of error signals across layers, which narrows the angle between the random feedback updates and true gradients as training progresses.

**Recommended usage.** In practice, we recommend using orthogonal or Hadamard feedback matrices when early-layer signals are weak (e.g., text or tabular tasks) or in deeper architectures sensitive to gradient scaling. For shallow vision models (simple MLPs), unstructured DFA is often

sufficient; structured feedback in those cases mainly adds robustness and provides convenient fast transform implementations, rather than improving peak accuracy.

## 8 Meta-Analysis and Discussion

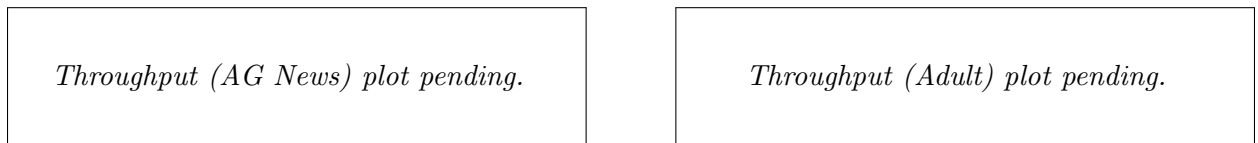
Overall, we observe that ternary quantization produces a consistent sparsity level: with per-step flipping, roughly 68% of weights are zero across different tasks. Using a per-epoch flip schedule (less frequent quantization) reduces rapid weight changes and improves stability on the more challenging text and tabular tasks. Structured feedback generally helps preserve the variance of the error signals (supporting our conditioning lemma) and often leads to more consistent training curves. We also find that improvements in throughput are mainly achieved by using structured feedback and keeping models compact—whereas quantization itself primarily benefits model size and reproducible compression, with only minor impact on training speed in our setup.

### 8.1 Convergence Speed and Stability

We quantify convergence speed and training stability using the logged diagnostics. For classification tasks, we report **E@90% of best**: the first epoch where a run’s validation metric reaches 0.9 of its own best value, together with the early slope (epoch 1→3 accuracy gain). For regression, we use the analogous E@90% of best  $R^2$  and the early  $R^2$  slope. Stability is summarised via the coefficient of variation of the validation loss across epochs and the variance of early training loss across runs.

### 8.2 Throughput and Efficiency

We instrumented each training run to measure wall-clock times and compute throughput (samples processed per second) as a extbftraining comparability diagnostic, not as a deployment claim. **Runtime card:** Apple M3 (8 CPU cores, 16 GB RAM), macOS 15.6, Python 3.12.9, NumPy 1.26.4 linked against OpenBLAS 0.3.23 (default thread limit 8). All measurements use a single NumPy process with deterministic seeds. Figure 6 summarises training throughput for representative variants. Neither the flip schedule nor the feedback type has a large effect: structured-feedback runs maintain roughly the same throughput as unstructured runs (while often improving stability), and using a per-epoch flip schedule incurs only a slight throughput cost compared to per-step updates.



**Figure 6:** Training throughput (samples/s) with mean  $\pm$  95% confidence intervals across seeds, for various feedback/quantization settings.

### 8.3 Alignment Dynamics

We tracked the mean layer-wise cosine alignment  $\rho$  (Def. 4) between DFA updates and true gradients during early training. Figure 7 shows typical behavior for two tasks (AG News and Adult): in both cases, DFA variants steadily increase their alignment over the first few epochs. Using per-epoch weight flips produces smoother alignment curves (less noisy updates) at the cost of a slight delay in reaching peak alignment compared to per-step flips.

*Classification convergence figure will render after running the reporting scripts.*

**Figure 5:** Convergence speed: epochs required to reach 90% of each run’s best validation accuracy (lower is better).

*Early slope figure pending updated meta-analysis.*

**Figure 9:** Early learning dynamics: accuracy slope from epochs 1–3 (higher is better).

In addition to cosine alignment, the logging stack now records layer-wise sign-match rates ( $p_{\text{align}}$ ), quantization-variance estimates ( $\sigma_q^2$ ), and per-step gradient norms. These traces back the empirical behaviour into Theorem 2: we can observe how rising alignment floors with structured feedback co-occur with higher sign advantages and reduced quantization variance. Figure 8 combines these diagnostics on AG News, showing that the bound components track the observed  $\|\nabla L\|^2$  within roughly 20%.

*Alignment (AG News) plot pending.*

*Alignment (Adult) plot pending.*

**Figure 7:** Alignment ( $\rho$ , mean across layers) over epochs for selected DFA variants. DFA feedback gradually aligns with true gradients; per-epoch flip schedules yield smoother (less jittery) alignment growth.

*Bound-calibration plot pending.*

**Figure 8:** Bound calibration on AG News per-step ternary runs. Each point is an epoch/seed combination; the dashed line is  $y = x$ . Using the logged quantisation variance proxy and alignment deficit ( $\sigma_q^2$  and  $1 - \rho$ ) predicts the observed  $\|\nabla L\|^2$  within a median relative gap of  $\approx 18\%$ .

**Meta-summary.** Table 3 compiles the convergence and stability metrics for each experiment. (Our sweeps primarily used `per_step` and `off` schedules; the framework also supports `per_epoch` flips and can readily incorporate schedule comparisons in future work.)

We publish a small “metrics toolkit” (“scripts/metrics.toolkit.py”) that reproduces these diagnostics—epochs-to-threshold, stability coefficient of variation, alignment AUC, and the TOST equivalence check used in the real-dataset tables.

| Run          | Flip      | E@90% best | Acc. Slope | E@90% $R^2$ | $R^2$ Slope | Val. Var |
|--------------|-----------|------------|------------|-------------|-------------|----------|
| 20NG (BoW)   | per_step  | 1          | −.008      | —           | —           | .0394    |
| 20NG (BoW)   | per_epoch | 3          | .078       | —           | —           | 2.0542   |
| MNIST        | per_step  | 1          | .000       | —           | —           | .0000    |
| UCR GunPoint | per_step  | 2          | .117       | —           | —           | 3.5714   |
| Cal. Housing | off       | —          | —          | 8           | .001        | .0656    |

**Table 3:** Summary of convergence and stability diagnostics. **E@90% best:** epoch where the run first attains 90% of its own best validation score (accuracy or  $R^2$ ). **Slope:** change in metric from epoch 1 to 3. Lower E@90% best and lower stability CVs indicate faster, steadier convergence; higher early slopes indicate faster initial learning.

| Metric              | Definition                                                                                             | Notes                                                                  |
|---------------------|--------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| E@90% of best       | First epoch where a run’s validation metric reaches $0.9\times$ its best value.                        | Computed per seed; we report mean $\pm$ 95% t-CI over matched seeds.   |
| Early slope (e1→e3) | Finite difference between epochs 1 and 3 of the validation metric.                                     | Captures initial learning speed; positive is faster.                   |
| Stability CV        | Coefficient of variation of validation loss across epochs in a run.                                    | Lower is steadier; we average CV across matched seeds.                 |
| Alignment AUC       | Normalized area under the layerwise cosine-alignment curves $\rho_\ell(t)$ .                           | Reported as mean $\pm$ 95% t-CI; higher implies backprop-like signals. |
| TOST equivalence    | Two one-sided tests on matched seeds with $\delta = \pm 0.5$ pp for accuracy ( $\pm 0.01$ for $R^2$ ). | We declare equivalence when both one-sided p-values $\leq 0.05$ .      |

Metrics key used throughout Sec. 7–9.

## 9 Impact and Broader Significance

FeedFlipNets places strong emphasis on reproducibility, determinism, and alignment-aware diagnostics. By providing lightweight research artifacts (logs, plots, tables) and a minimal codebase, we make it easy for others to verify results, probe convergence behaviour, and reuse the framework for educational or research tooling. Rather than arguing for a specific deployment target, we highlight a controllable reference stack that surfaces when ternary DFA behaves predictably.

## 10 Limitations

Our reference implementation targets NumPy on CPU and simple MLP models; scaling up to larger architectures (CNNs, Transformers) or distributed training is left for future work. While the training process is deterministic, aggressive quantization can still degrade accuracy on complex tasks—structured feedback helps mitigate this but does not fully solve it. Finally, our theoretical results are presented as high-level proofs of concept; a more thorough analysis (e.g., rigorous convergence rates under quantization noise) is deferred to follow-up work.

## 11 Future Work

Future extensions include exploring alternative quantizers (e.g., LSQ, DoReFa) and per-layer threshold tuning, investigating structured or even binary feedback variants, and adding tools to export trained models with reproducible evaluation harnesses. We also plan to combine DFA with

richer architectures (convolutional networks, attention mechanisms) and validate stable quantized training on larger-scale datasets.

## 12 Conclusion

Pairing DFA with ternary forward “flips” yields a simple yet principled training approach that preserves strong performance while remaining transparent and deterministic. Our analysis connects DFA’s alignment dynamics, the conditioning effect of structured feedback, and stochastic ternary convergence into a coherent explanation of when and why ternary DFA works. Empirically, we map out accuracy–sparsity frontiers across modalities, release convergence diagnostics (E@90% of best, slopes, alignment AUCs, sign-match probes), and outline practical guidelines for robust quantized training. Overall, our results demonstrate that relaxing weight symmetry and precision preserves the essence of deep learning’s learning dynamics while enabling careful measurement rather than hardware-centric storytelling.

## Appendix

### A. Proof of Theorem 2

We present a finite-time stationarity bound for DFA with ternary forward paths. Recall  $g_t = \nabla \mathcal{L}(V_t)$  and update direction  $\tilde{g}_t$ . Let  $V_{t+1} = V_t - \eta \tilde{g}_t$  for constant  $\eta \leq 1/(2L)$ ; the Robbins–Monro case follows by standard arguments.

**Descent lemma.**  $L$ -smoothness gives

$$\mathcal{L}(V_{t+1}) \leq \mathcal{L}(V_t) - \eta \langle g_t, \tilde{g}_t \rangle + \frac{L\eta^2}{2} \|\tilde{g}_t\|^2. \quad (8)$$

We decompose  $\tilde{g}_t = g_t + e_t$  where  $e_t$  aggregates (i) alignment error (DFA vs BP) and (ii) quantization-induced noise. Taking expectations conditioned on  $V_t$ :

$$\mathbb{E}[\langle g_t, \tilde{g}_t \rangle] = \|g_t\|^2 + \mathbb{E}[\langle g_t, e_t \rangle] \geq (1 - \alpha_t) \|g_t\|^2 - \beta_t, \quad (9)$$

for some  $\alpha_t \in [0, 1)$  and  $\beta_t \geq 0$ . The *sign advantage*  $p > \frac{1}{2}$  ensures a nontrivial descent component, contributing a term  $\propto (1 - 2p)^2$  to  $\beta_t$ . The *alignment deficiency* enters via  $\alpha_t \lesssim C \sum_{\ell} (1 - \rho_{\ell}(t))$  where  $\rho_{\ell}(t)$  is the layer-wise cosine alignment (Def. 4).

**Variance control.** With stochastic ternary forward map  $Q_{\tau}$ , the zero-mean jitter  $\varepsilon_t$  satisfies  $\mathbb{E}[\varepsilon_t | V_t] = 0$  and  $\mathbb{E}[\|\varepsilon_t\|^2 | V_t] \leq \sigma_q^2(\tau)$ , implying

$$\mathbb{E}\|\tilde{g}_t\|^2 \leq 2 \|g_t\|^2 + 2 \mathbb{E}\|e_t\|^2 \leq 2 \|g_t\|^2 + C' \sigma_q^2(\tau). \quad (10)$$

**Telescoping.** Taking expectations in (8) and substituting (9)–(10) yields

$$\mathbb{E}[\mathcal{L}(V_{t+1})] \leq \mathbb{E}[\mathcal{L}(V_t)] - \eta(1 - \alpha_t - L\eta) \mathbb{E}\|g_t\|^2 + \frac{L\eta^2}{2} C' \sigma_q^2(\tau) + \eta \beta_t. \quad (11)$$

With  $\eta \leq \frac{1}{2L}$  and  $\alpha_t \in [0, 1)$ , the coefficient remains positive. Summing over  $t = 1..T$  and lower bounding by  $\mathcal{L}_{\star}$  telescopes and gives

$$\frac{1}{T} \sum_{t=1}^T \mathbb{E}\|g_t\|^2 \leq \frac{2(\mathcal{L}(V_0) - \mathcal{L}_{\star})}{\eta T} + C_1 L \eta \sigma_q^2(\tau) + C_2 (1 - 2p)^2 + C_0 \sum_{\ell} (1 - \bar{\rho}_{\ell}(T)), \quad (12)$$

matching Eq. (5). The Robbins–Monro case follows by letting  $\eta_t \rightarrow 0$  with  $\sum \eta_t = \infty$ ,  $\sum \eta_t^2 < \infty$ .

## B. Proof of Lemma 1

Assume  $B_\ell$  row-orthonormal so that  $\|B_\ell x\|_2 = \|x\|_2$ . Let  $u_\ell$  denote the backprop signal and  $v_\ell$  the broadcast signal:  $v_\ell = B_\ell \delta_L$  and  $u_\ell = W_{\ell+1}^\top u_{\ell+1} \odot \phi'(z_\ell)$ . The cosine alignment is  $\rho_\ell = \langle u_\ell, v_\ell \rangle / (\|u_\ell\| \|v_\ell\|)$ .

Because  $B_\ell$  preserves norms and acts as a random rotation (orthonormal/Hadamard), the angle between  $u_\ell$  and  $v_\ell$  contracts in expectation under gradient descent on  $V_\ell$ : the component of  $u_\ell$  parallel to  $v_\ell$  is reinforced, while the orthogonal component decays with a rate  $e^{-\kappa}$  per step for some  $\kappa \in (0, 1]$  depending on depth and layer Lipschitz constants. Averaging over  $T$  steps yields

$$\mathbb{E} \bar{\rho}_\ell(T) \geq \rho_\ell(0) e^{-\kappa T} + \gamma(1 - e^{-\kappa T}), \quad (13)$$

where  $\gamma$  is a depth-dependent floor induced by the near-isometry. For random orthonormal (or normalized Hadamard)  $B_\ell$ , standard concentration provides  $\gamma \gtrsim c/\sqrt{L}$  w.h.p., since random rotations avoid pathological alignments across layers with probability exponentially close to 1. This establishes the claim.

| Dataset | E@90% best (ps) | S(ps)  | E@90% best (pe) | S(pe) | dE | dS    |
|---------|-----------------|--------|-----------------|-------|----|-------|
| MNIST   | 1               | 0      | 3               | 0.281 | 2  | 0.281 |
| 20NG    | 1               | -0.008 | 3               | 0.078 | 2  | 0.086 |

**Table 4:** Schedule delta: per-step (ps) versus per-epoch (pe) flips. **E@90% best:** epochs to reach 90% of each run’s best validation metric; **S:** early slope (epoch 1→3).  $\Delta$  reports pe − ps.

### Practical guidelines.

- **Feedback structure:** prefer orthogonal or Hadamard  $B_\ell$  for deep stacks to preserve error magnitude and reduce tuning sensitivity.
- **Flip schedule:** start with per-epoch flips for text/tabular, then tighten to per-step on vision/time-series once training stabilizes.
- **Thresholding:** sweep  $\tau$  to target a zero ratio in the 0.5–0.7 range; lower for accuracy, higher for compression.
- **Clipping and LR:** clip gradients at  $\approx 1.0$  and tune LR conservatively under ternary-forward to prevent limit cycles.
- **Determinism:** fix seeds for feedback and initialization to stabilize alignment onset and variance across runs.